

Notification Framework - Technical Documentation

1. Project Overview

The Notification Framework is a multi-channel notification platform. It lets a registered client send notifications to users through email, SMS, WhatsApp, and push channels while enforcing preferences, DND windows, quotas, rate limits, queue controls, provider failover, media attachment handling, delivery logs, analytics, and dead-letter-queue recovery.

The project contains these main parts:

Component	Path	Purpose
Notification API	<code>notification_service/main.py</code>	FastAPI service that accepts notification requests, manages clients, preferences, providers, queues, logs, analytics, media, and webhooks.
Notification worker	<code>notification_service/tasks.py</code>	Celery worker that consumes queued jobs from RabbitMQ and sends them through configured providers.
DLQ processor	<code>notification_service/dlq_processor.py</code>	Background processor that retries or drops dead-letter jobs based on error type.
Auth/API-key server	<code>login_registration/server/index.js</code>	Express service for registration, login, token/key lifecycle, and syncing API keys into FastAPI.
Client/admin dashboard	<code>login_registration/client</code>	React dashboard for signup/login, API keys, jobs, queues, DLQ, analytics, clients, and providers.
Notification test UI	<code>notification_service/src</code>	React UI for sending JSON notification requests and managing user preferences.
Infrastructure	<code>docker-compose.yml</code>	Runs MongoDB, RabbitMQ, FastAPI, Celery worker, auth server, and both frontend apps.

2. Technology Stack

Layer	Technology
API	Python 3.11, FastAPI, Pydantic, Uvicorn
Worker	Celery, Kombu
Broker	RabbitMQ
Database	MongoDB
Auth server	Node.js, Express, Mongoose, bcrypt, JWT
Frontend	React, Vite, Axios, Bootstrap
Providers	SendGrid, SMTP, SendFire, UltraMsg, Firebase FCM

3. Runtime Architecture

High-level flow:

1. A company registers through the React dashboard or directly through the auth server.
2. The auth server creates a `client_id` and an `nf_...` API key.
3. The auth server stores a bcrypt hash in `api_clients` and syncs the plain key once to FastAPI `/clients/sync`.
4. FastAPI stores a SHA-256 hash in the `clients` collection.
5. The client calls `POST /notify` with `X-API-Key`.
6. FastAPI validates the key, checks the client ID, event scope, allowed channels, quota, user preferences, DND windows, character limits, attachments, and rate limits.
7. FastAPI creates one channel-level job per resolved channel and publishes it to the active RabbitMQ queue.
8. Celery workers consume jobs from queues such as `email-notify-q`, `sms-notify-q`, `whatsapp-notify-q`, and `push-notify-q`.
9. Workers send through the configured provider snapshot, retry transient failures, switch providers when configured, write delivery logs, and push terminal failures to the DLQ.
10. Provider webhooks update delivery status, read status, latency, and analytics.

4. Services and Ports

When using `docker-compose.yml`:

Service	URL/Port	Notes
MongoDB	localhost:27017	Database name: <code>notification_db</code> .
RabbitMQ AMQP	localhost:5672	Celery broker.
RabbitMQ UI	http://localhost:15672	Default credentials: <code>guest / guest</code> .
Notification API	http://localhost:8000	FastAPI app and Swagger docs at <code>/docs</code> .
Auth server	http://localhost:3001	Express login/registration API.
Login dashboard	http://localhost:5173	React client/admin dashboard.
Notification test UI	http://localhost:5174	JSON sender and preferences UI.

5. Environment Configuration

Create `notification_service/.env` from `notification_service/.env.example`.

Important variables:

Variable	Purpose
MONGO_URL	MongoDB connection string. Docker uses <code>mongodb://mongodb:27017</code> ; local usually uses <code>mongodb://localhost:27017</code> .
CELERY_BROKER_URL	RabbitMQ broker URL. Docker uses <code>amqp://guest:guest@rabbitmq:5672//</code> .
PUBLIC_BASE_URL	Public API base URL used for media links. For WhatsApp local testing, use a public HTTPS tunnel such as ngrok.
MEDIA_DIR	Directory for uploaded local files. Defaults to <code>notification_service/media_store</code> .
MEDIA_URL_TTL_DAYS	Signed media URL validity window.
SENDGRID_API_KEY, SENDGRID_FROM	SendGrid email provider credentials.
SMTP_EMAIL, SMTP_PASSWORD	SMTP fallback credentials.
SENDFIRE_API_KEY, SENDFIRE_SENDER_ID	SMS provider credentials.
ULTRMSG_INSTANCE_ID, ULTRMSG_TOKEN	WhatsApp provider credentials.
FCM_PROJECT_ID, FCM_SERVICE_ACCOUNT_FILE, FCM_SERVICE_ACCOUNT_JSON	Firebase Cloud Messaging configuration.
SENDGRID_WEBHOOK_KEY	Optional SendGrid webhook signature verification key.
ULTRMSG_WEBHOOK_TOKEN	Optional UltraMsg webhook verification token.
QUEUE_FAILOVER_ENABLED	Enables automatic primary/backup queue switching.
PROVIDER_CAP_*_JOBS_PER_SEC	Provider capacity caps used by effective rate limiting.

For the Node auth server, Docker injects:

Variable	Purpose
MONGO_URL	MongoDB database URL for <code>api_clients</code> .
FASTAPI_URL	FastAPI URL used by Node to sync API keys.

6. Setup With Docker Compose

From the repository root:

```
docker compose up --build
```

Then open:

```
Login dashboard:      http://localhost:5173
Notification test UI:  http://localhost:5174
FastAPI Swagger docs: http://localhost:8000/docs
RabbitMQ dashboard:   http://localhost:15672
```

Stop the stack:

```
docker compose down
```

Stop and delete volumes/data:

```
docker compose down -v
```

7. Manual Local Setup

Use this if you do not want Docker for app services.

1. Start MongoDB and RabbitMQ locally.
2. Create `notification_service/.env`.
3. Install and run FastAPI:

```
cd notification_service
pip install fastapi uvicorn pymongo python-dotenv celery kombu requests python-multipart google-auth
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

4. In another terminal, start the worker:

```
cd notification_service
celery -A tasks worker --loglevel=info --concurrency=4 -Q email-notify-q,sms-notify-q,whatsapp-notify-q,push-notify-q
```

5. Start the auth server:

```
cd login_registration/server
npm install
npm start
```

6. Start the login dashboard:

```
cd login_registration/client
npm install
npm run dev -- --host 0.0.0.0 --port 5173
```

7. Start the notification test UI:

```
cd notification_service
npm install
npm run dev -- --host 0.0.0.0 --port 5174
```

8. How To Use The Project Step By Step

Step 1: Register a Client

Use the login dashboard at <http://localhost:5173/register>.

You will receive:

Value	Meaning
<code>client_id</code>	The client identifier that must be used in <code>/notify</code> requests.
<code>api_key</code>	The <code>nf_...</code> API key. It is shown once. Use it as the <code>X-API-Key</code> header.
JWT token	Used by the dashboard for protected Node routes.

Alternative direct API call:

```
curl -X POST http://localhost:3001/register \
-H "Content-Type: application/json" \
-d '{
  "name": "Demo Admin",
  "client_name": "Demo Company",
  "email": "demo@example.com",
  "password": "password123"
}'
```

Step 2: Save User Preferences

Open <http://localhost:5174/preferences> or call:

```
curl -X POST http://localhost:8000/preferences \
-H "Content-Type: application/json" \
-d '{
  "user_id": "user_1",
  "email": "user@example.com",
  "phone": "+919876543210",
  "wa_number": "+919876543210",
  "fcm_tokens": ["firebase-device-token"],
  "event_channel_preferences": [
    {
      "event_type": "PAYMENT_SUCCESS",
      "channels": ["email", "sms"],
      "dnd_mode": "none",
      "dnd_windows": []
    }
  ],
  "dnd_windows": [
    {
      "days": ["Mon", "Tue", "Wed", "Thu", "Fri"],
      "start": "22:00",
      "end": "07:00",
      "tz": "Asia/Kolkata"
    }
  ]
}'
```

Preferences are used during `/notify` to decide which channels are allowed for a user/event. If no preference exists, the API falls back to `channels_requested`.

Step 3: Configure Providers

Open the dashboard at <http://localhost:5173>, go to the providers tab, and enable providers per channel.

Direct API example:

```
curl -X POST http://localhost:8000/providers \
-H "Content-Type: application/json" \
-d '{
  "provider_id": "sendgrid",
  "provider_name": "SendGrid",
  "channel": "email",
  "priority": 1,
  "is_active": true,
  "max_retries": 3,
  "timeout_ms": 5000
}'
```

If the `providers` collection has no active provider for a channel, FastAPI uses built-in defaults:

Channel Default providers

email	SendGrid, SMTP
sms	SendFire
whatsapp	UltraMsg
push	Firebase FCM

Step 4: Send a Notification

Open `http://localhost:5174`, paste your X-API-Key, edit the JSON, and send.

Direct API example:

```
curl -X POST http://localhost:8000/notify \
-H "Content-Type: application/json" \
-H "X-API-Key: nf_your_api_key_here" \
-d '{
  "client_id": "your_client_id",
  "event_type": "PAYMENT_SUCCESS",
  "recipients": [
    {
      "user_id": "user_1",
      "email": "user@example.com",
      "phone": "+919876543210",
      "wa_number": "+919876543210",
      "fcm_token": "firebase-device-token"
    }
  ],
  "channels_requested": ["email"],
  "priority": "HIGH",
  "content": {
    "subject": "Payment received",
    "body": "Your payment was successful."
  }
}'
```

Successful response:

```
{
  "message": "Notification request processed",
  "request_id": "req...",
  "jobs_created": ["job..."],
  "jobs_blocked": []
}
```

Step 5: Track Jobs and Logs

Use the dashboard tabs or call:

```
curl "http://localhost:8000/jobs?client_id=your_client_id&limit=50"
curl "http://localhost:8000/logs?client_id=your_client_id&limit=50"
curl "http://localhost:8000/stats?client_id=your_client_id"
```

Step 6: Operate Queues

Queue admin endpoints require an `Authorization: Bearer <token>` header. The FastAPI side only checks that a bearer token is present; the dashboard obtains this token from the Node login flow.

Examples:

```
curl http://localhost:8000/queue-stats \
-H "Authorization: Bearer your_dashboard_jwt"

curl -X POST http://localhost:8000/pause-queue \
-H "Authorization: Bearer your_dashboard_jwt" \
-H "Content-Type: application/json" \
-d '{"queue": "email"}'

curl -X POST http://localhost:8000/resume-queue \
-H "Authorization: Bearer your_dashboard_jwt" \
-H "Content-Type: application/json" \
-d '{"queue": "email"}'

curl -X POST http://localhost:8000/update-rate-limit \
-H "Authorization: Bearer your_dashboard_jwt" \
-H "Content-Type: application/json" \
-d '{"queue": "email", "rate": 120}'
```

Step 7: Retry or Discard DLQ Jobs

```
curl "http://localhost:8000/dlq?status=DLQ&limit=50"

curl -X POST http://localhost:8000/retry-job \
-H "Authorization: Bearer your_dashboard_jwt" \
-H "Content-Type: application/json" \
-d '{"job_id": "job_..."}'

curl -X POST http://localhost:8000/discard-job \
-H "Authorization: Bearer your_dashboard_jwt" \
-H "Content-Type: application/json" \
-d '{"job_id": "job_..."}'
```

9. Notification Request Model

POST /notify requires:

Field	Type	Required	Description
client_id	string	Yes	Must match the API-key owner.
event_type	string	Yes	Event name such as <code>PAYMENT_SUCCESS</code> . Event-scoped API keys can only send their own event type.
recipients	array	Yes	List of recipient objects.

Field	Type	Required	Description
channels_requested	array	No	Requested channels. If omitted, preference resolution decides fallback behavior.
priority	string	No	Stored on the job. Defaults to <code>MEDIUM</code> .
content	object	No	Message payload. Fields depend on channel.

Recipient fields:

Field	Used by
user_id	Required for preference, DND, logs, rate limits.
email	Email channel.
phone	SMS channel.
wa_number	WhatsApp channel.
fcm_token	Push channel.

Content fields:

Channel	Common content fields
Email	subject, body or message, attachments
SMS	body or message
WhatsApp	body or message, attachments
Push	title or subject, body, optional data fields

10. Media Attachments

Large files should not be embedded in `/notify`. Upload or register media first, then use `file_id` in `content.attachments`.

Upload local file:

```
curl -X POST http://localhost:8000/media/upload \
  -H "X-API-Key: nf_your_api_key_here" \
  -F "file=@/path/to/file.pdf" \
  -F "delivery_mode=auto"
```

Register remote URL:

```
curl -X POST http://localhost:8000/media/upload \
  -H "X-API-Key: nf_your_api_key_here" \
  -F "remote_url=https://example.com/file.pdf" \
  -F "name=file.pdf" \
  -F "mime_type=application/pdf" \
  -F "delivery_mode=link_only"
```

Use in notification payload:

```
{
  "content": {
    "subject": "Document ready",
    "body": "Please see the attachment.",
    "attachments": [
      {
        "file_id": "mf_..."
      }
    ]
  }
}
```

Important WhatsApp note: WhatsApp providers fetch media from their own servers. Localhost URLs are not public, so for local WhatsApp attachment testing set `PUBLIC_BASE_URL` to a public HTTPS URL.

11. API Authentication

There are two authentication styles:

Use case	Header
Client notification APIs	X-API-Key: nf_...
Dashboard/admin APIs	Authorization: Bearer <dashboard JWT>

FastAPI stores only the SHA-256 hash of the nf_... key in `clients`.

Node stores bcrypt hashes in `api_clients`. Because bcrypt cannot be used for direct FastAPI lookup, Node syncs the plain key once to `POST /clients/sync`, where FastAPI stores its SHA-256 hash.

12. API Endpoint Reference

Client and API Key Endpoints

Method	Endpoint	Purpose
POST	<code>/clients/register</code>	Register API client directly in FastAPI and return one-time API key.
POST	<code>/clients/sync</code>	Upsert client key from Node auth server.
GET	<code>/clients</code>	List registered clients without key hashes.
PATCH	<code>/clients/{client_id}</code>	Update client settings.
POST	<code>/clients/{client_id}/rotate-key</code>	Rotate FastAPI API key directly.
DELETE	<code>/clients</code>	Delete all clients and scoped data. Requires bearer token.
DELETE	<code>/clients/{client_id}</code>	Delete a specific client and scoped data. Requires bearer token.

Notification and Preference Endpoints

Method	Endpoint	Purpose
POST	<code>/notify</code>	Validate and queue notifications. Requires X-API-Key.
POST	<code>/preferences</code>	Create/update user preferences and DND windows.
GET	<code>/preferences/{user_id}</code>	Fetch preferences.
GET	<code>/user-event-types/{user_id}</code>	List event types seen for a user.
GET	<code>/unsubscribe</code>	Remove email channel for a user/event.

Media Endpoints

Method	Endpoint	Purpose
POST	<code>/media/upload</code>	Upload local file or register remote URL. Requires X-API-Key.
GET	<code>/media/{file_id}</code>	Serve/redirect media through owner auth or signed token.

Queue and Rate Limit Endpoints

Method	Endpoint	Purpose
GET	<code>/queue-stats</code>	Queue depths, active jobs, failed jobs, failover state.
POST	<code>/pause-queue</code>	Pause a channel queue.
POST	<code>/resume-queue</code>	Resume a channel queue.
POST	<code>/update-rate-limit</code>	Set channel jobs/min limit.
POST	<code>/clear-queue</code>	Purge waiting messages and mark queued jobs as cleared.
GET	<code>/rate-limit/effective/{channel}</code>	Show effective per-user/client limits.
GET	<code>/rate-limit-status</code>	Show in-memory global rate limiter state.
GET	<code>/rate-limit-logs</code>	Query per-user/client/global rate-limit hits.
GET	<code>/global-rate-limit-logs</code>	Query global middleware rate-limit hits.

Logs, Analytics, and DLQ Endpoints

Method	Endpoint	Purpose
GET	<code>/jobs</code>	List job documents with derived dashboard fields.
GET	<code>/logs</code>	Query delivery logs.

Method	Endpoint	Purpose
GET	/stats	Aggregate delivery stats.
GET	/latency	Latency percentiles and trends.
GET	/failure-analytics	Failure and delivery analytics by event type.
GET	/dlq	Query dead-letter jobs.
POST	/retry-job	Retry a DLQ job.
POST	/discard-job	Mark one DLQ job as dropped.
POST	/discard-dlq	Bulk discard DLQ jobs.
POST	/retry-failed	Bulk retry transient DLQ entries.
POST	/reprocess-dlq	Bulk reprocess non-permanent DLQ entries.

Provider and Webhook Endpoints

Method	Endpoint	Purpose
GET	/providers	List provider configuration.
POST	/providers	Add/update provider configuration.
PUT	/clients/{client_id}/webhook-url	Configure client callback URL.
GET	/clients/{client_id}/webhook-url	Fetch configured client callback URL.
POST	/webhooks/sendgrid	Receive SendGrid delivery events.
POST	/webhooks/ultramsq	Receive UltraMsg delivery/read events.
GET	/webhooks/ultramsq/latest	Debug latest UltraMsg inbound payload.

Node Auth Server Endpoints

Method	Endpoint	Purpose
POST	/register	Create user/client and default API key.
POST	/login	Login and receive dashboard JWT.
GET	/home	Fetch client ID, event tokens, quota data. Requires JWT.
POST	/create-token	Create event-scoped API key. Requires JWT.
POST	/refresh-token	Rotate event token. Requires JWT.
POST	/disable-token	Disable event token. Requires JWT.
POST	/delete-disabled-token	Delete disabled token. Requires JWT.
POST	/sync-to-fastapi	Manually sync pasted API key to FastAPI. Requires JWT.
POST	/generate-api-key	Regenerate default API key and sync to FastAPI. Requires JWT.
DELETE	/delete-account	Delete account and scoped notification data. Requires JWT.

13. Queue Design

The system uses one primary RabbitMQ queue per channel:

Channel	Queue
Email	email-notify-q
SMS	sms-notify-q
WhatsApp	whatsapp-notify-q
Push	push-notify-q

Queue control state is stored in `queue_controls`.

Important fields:

Field	Description
<code>paused</code>	If true, workers requeue jobs later instead of sending.
<code>rate_limit</code>	Admin jobs/min throttle. 0 means unset.
<code>active_queue</code>	Queue currently used for new jobs.
<code>backup_queue</code>	Dynamic backup queue used during congestion/failover.
<code>backup_enabled</code>	Enables/disables backup queue use.

FastAPI runs a queue failover monitor in the API process. It watches primary queue depth and age signals. When primary queues are congested or stuck, the API can switch active dispatch to a backup queue and later recover to the primary queue.

14. Rate Limiting and Quotas

The framework has multiple protection layers:

Layer	Location	Description
Global per-IP	FastAPI middleware	In-memory limit of requests per IP per window.
Global system	FastAPI middleware	In-memory total API request cap per window.
Monthly quota	/notify	Counts monthly jobs per client.
Per-user/channel	/notify	Rolling count by user and channel.
Per-client/channel	/notify	Rolling count by client and channel.
Admin channel throttle	Worker	Jobs/min throttle per channel from <code>queue_controls</code> .
Provider cap	API config	Prevents admin throttles from exceeding provider capacity.

Default request middleware limit:

Scope	Limit
Per IP	5000 requests / 60 seconds
System	5000 requests / 60 seconds

Default notification rate limits:

Channel	Per-user	Per-client
Email	5000 / 60s	5000 / 60s
SMS	5 / 3600s	500 / 3600s
WhatsApp	5 / 60s	500 / 3600s
Push	50 / 3600s	5000 / 3600s

15. Provider Failover and Retry Logic

Provider configuration is captured into each job as `providers_snapshot`. This makes job behavior stable even if provider settings are edited later.

Worker behavior:

1. Try provider at `retry_meta.provider_index`.
2. If it succeeds, write `delivery_logs` with status `SENT` and update the job.
3. If it fails and attempts remain, retry the same provider with backoff.
4. If attempts are exhausted, switch to the next provider.
5. If all providers fail, create a DLQ entry and mark the job `DLQ`.

Error classification:

Error code	Type	Retry behavior
<code>INVALID_EMAIL</code>	Permanent	Drop in DLQ processor.
<code>USER_UNSUBSCRIBED</code>	Permanent	Drop.
<code>INVALID_PAYLOAD</code>	Permanent	Drop.
<code>SPAM_REJECTED</code>	Permanent	Drop.
<code>AUTH_FAILED</code>	Permanent	Drop.
<code>RATE_LIMIT_EXCEEDED</code>	Transient	Retry later.
<code>SMTP_TIMEOUT</code>	Transient	Retry and switch provider.
<code>UNKNOWN_ERROR</code>	Transient	Retry later.

16. MongoDB Collections

Collection	Purpose
<code>api_clients</code>	Node auth users/clients and bcrypt-hashed event tokens.
<code>clients</code>	FastAPI clients with SHA-256 API-key hash and allowed channels.
<code>notification_requests</code>	Raw incoming notification requests and request status.
<code>notification_jobs</code>	Queued/processing/sent/DLQ job records.
<code>delivery_logs</code>	Per-recipient provider outcome logs.

Collection	Purpose
user_preferences	User contacts, event channel preferences, DND windows.
rate_limit_logs	Per-user/per-client notification rate-limit hits.
global_rate_limit_logs	Global middleware rate-limit hits.
providers	Dynamic provider configuration.
queue_controls	Pause/resume/rate/failover state per channel.
queue_rate_counters	Worker-side per-minute channel throttle counters.
dlq	Dead-letter entries and retry/drop metadata.
webhook_calls	Client webhook callback delivery attempts.
ultramsg_inbound	Raw UltraMsg webhook payloads for debugging.
media_files	Uploaded or remote media metadata.

17. Delivery Status Lifecycle

Common job statuses:

Status	Meaning
RECEIVED	Notification request stored.
QUEUED	Job created and published to RabbitMQ.
PROCESSING	Worker is handling the job.
SENT	Provider accepted the message.
DELIVERED	Provider webhook confirmed delivery.
READ	Provider webhook confirmed read/open.
FAILED	Provider failure recorded.
DLQ	All retries/providers exhausted.
REQUEUED	DLQ job was scheduled again.
DROPPED	DLQ job was discarded or deemed permanent.
CLEARED	Queued job was cleared by queue admin action.

18. Webhooks

Provider Webhooks

SendGrid:

```
POST /webhooks/sendgrid
```

Handles events such as delivered, open, bounce, and spamreport.

UltraMsg:

```
POST /webhooks/ultramsg
```

Handles sent, delivered, read, and failed events from varied UltraMsg payload shapes.

Client Callback Webhooks

Clients can configure a webhook URL:

```
curl -X PUT http://localhost:8000/clients/your_client_id/webhook-url \
  -H "Authorization: Bearer your_dashboard_jwt" \
  -H "Content-Type: application/json" \
  -d '{"webhook_url": "https://example.com/notification-callback"}'
```

The worker and webhook handlers emit best-effort events shaped like:

```
{
  "type": "NOTIFICATION_STATUS",
  "job_id": "job...",
  "request_id": "req...",
  "client_id": "client...",
  "event_type": "PAYMENT_SUCCESS",
  "channel": "email",
  "status": "SENT",
  "provider": "SendGrid",
  "provider_message_id": "...",
  "timestamp": "..."
}
```

19. Character and Payload Limits

FastAPI validates content before queueing:

Channel	Limit
Email subject	Hard limit 998 chars, warning over 78 chars.
Email body	100,000 chars.
SMS body	160 chars per segment, max 3 segments / 480 chars.
WhatsApp body	4096 chars.
Push title	65 chars.
Push body	240 chars.
Media upload	Default 250 MB, configurable with <code>MEDIA_UPLOAD_MAX_MB</code> .

20. Development Notes

Useful commands:

```
# List all project files
rg --files

# Start complete stack
docker compose up --build

# Follow a single container
docker compose logs -f notification-api
docker compose logs -f notification-worker
docker compose logs -f auth-server

# Build frontend bundles
cd login_registration/client && npm run build
cd notification_service && npm run build
```

Swagger docs are available at:

```
http://localhost:8000/docs
```

21. Common Troubleshooting

Problem	Likely cause	Fix
<code>/notify</code> returns 401 Invalid API key	Key was not synced into FastAPI or wrong key was pasted.	Regenerate/sync from dashboard, then use the new <code>nf_...</code> key.

Problem	Likely cause	Fix
<code>/notify</code> returns 403 <code>client_id</code> does not match	Payload <code>client_id</code> belongs to a different API key.	Use the <code>client_id</code> shown for the API key owner.
<code>/notify</code> returns event-scope error	Event-specific key is being used for another event.	Use a <code>DEFAULT</code> key or the key for that event type.
Jobs stay queued	Worker is not running, RabbitMQ is down, or queue is paused.	Check <code>notification-worker</code> , RabbitMQ UI, and <code>/queue-stats</code> .
WhatsApp attachments fail locally	Provider cannot fetch localhost media URL.	Set <code>PUBLIC_BASE_URL</code> to a public HTTPS tunnel.
Provider auth failures	Missing or wrong <code>.env</code> provider credentials.	Fill provider-specific environment variables.
DLQ fills up	Provider failures, invalid payloads, rate limits, or auth errors.	Inspect <code>/dlq</code> , <code>delivery_logs</code> , and provider credentials.
Rate-limit 429	Global, per-user/client, quota, or admin throttle exceeded.	Inspect <code>Retry-After</code> , <code>/rate-limit-logs</code> , and <code>/rate-limit/effective/{channel}</code> .

22. Recommended Production Hardening

Before production use:

1. Replace the hardcoded Node `JWT_SECRET` with a strong environment variable.
2. Restrict FastAPI CORS instead of `allow_origins=["*"]`.
3. Protect FastAPI admin endpoints with real JWT verification, not bearer-token presence only.
4. Move global rate-limit counters from in-memory dictionaries to Redis for multi-process deployments.
5. Store provider credentials in a secret manager.
6. Do not commit real `firebase-service-account.json` or `.env` files.
7. Put MongoDB and RabbitMQ behind authenticated private networking.
8. Add indexes on high-query collections such as `notification_jobs`, `delivery_logs`, `dlq`, and `clients`.
9. Run API and workers with process managers or orchestrators that restart on failure.
10. Configure webhook signatures/tokens for provider webhooks.